

Minor Exam

● Graded

Student

Chirag Kathpalia

Total Points

32.5 / 90 pts

Question 1

Problem 1

8.5 / 12 pts

1.1 (I)

1.5 / 2 pts

+ 2 pts (a), (c) and (d)

✓ + 1.5 pts any 2 of (a), (c) and (d)

+ 1 pt any 1 of (a), (c) and (d)

+ 0 pts Incorrect / Unattempted

1.2 (II)

4 / 6 pts

✓ + 1 pt selected (a)

✓ + 1 pt selected (c)

+ 2 pts justified not selecting (b) correctly
UTXO fully spent; change output created.

✓ + 2 pts justified not selecting (d) correctly
UTXO model, not account state.

+ 0 pts Incorrect / Unattempted

1.3 (III)

3 / 4 pts

✓ + 1 pt selected (a)

✓ + 1 pt selected (c)

+ 1 pt justify not selecting (b) correctly
Consensus based on majority of mining power, not majority of nodes.

✓ + 1 pt justify not selecting (d) correctly
Private keys allow spending, not rewriting confirmed history.

+ 0 pts Incorrect / Unattempted

Question 2

Problem 2

8 / 16 pts

2.1 (I)

4 / 4 pts

✓ + 2 pts B decrypts -> Obtains valid signature on m

✓ + 2 pts B forwards signed m to C
 B sends $E_{PK_C}(S_{SK_A}(m))$ to C

+ 0 pts Incorrect / Unattempted

2.2 (II)

4 / 4 pts

✓ + 2 pts Signature binds ciphertext; Ciphertext bound to C

✓ + 2 pts C cannot decrypt forwarded message

+ 0 pts Incorrect / Unattempted

2.3 (III)

0 / 8 pts

+ 2 pts Use malleability

+ 2 pts Construct new ciphertext

+ 2 pts Oracle query

+ 2 pts Recovery of m_b

✓ + 0 pts Incorrect / Unattempted

3.1 (I)

1 / 3 pts

+ 3 pts Any 3 of the following:

1. Multiple sigs vs single aggregated
2. Linear signature size vs constant
3. Reveals signers vs hides subset
4. Larger on-chain cost vs lower
5. Independent pub-key vs distributed key gen produces one pub-key

+ 2 pts Any 2 of the following:

1. Multiple sigs vs single aggregated
2. Linear signature size vs constant
3. Reveals signers vs hides subset
4. Larger on-chain cost vs lower
5. Independent pub-key vs distributed key gen produces one pub-key

✓ + 1 pt Any 1 of the following:

1. Multiple sigs vs single aggregated
2. Linear signature size vs constant
3. Reveals signers vs hides subset
4. Larger on-chain cost vs lower
5. Independent pub-key vs distributed key gen produces one pub-key

+ 0 pts Incorrect / Unattempted

3.2 (II)

0 / 3 pts

+ 1 pt Degree = 2**+ 1 pt** Mentioned 3 points to interpolate.**+ 1 pt** 2 shares give infinite possibilities.**✓ + 0 pts** Incorrect / Unattempted

3.3 (III)

1 / 4 pts

✓ + 1 pt Use sorted tree**+ 1 pt** Find neighbouring leaves**+ 1 pt** Provide inclusions proofs**+ 1 pt** Show gap excludes element**+ 0 pts** Incorrect / Unattempted

Question 4

Problem 4

5 / 12 pts

4.1 (I)

1 / 3 pts

- + 3 pts** Any 3 of the following:
1. Synchronous system
 2. Reliable delivery
 3. No message forgery
 4. Known sender identity

- + 2 pts** Any 2 of the following:
1. Synchronous system
 2. Reliable delivery
 3. No message forgery
 4. Known sender identity

- ✓ **+ 1 pt** Any 1 of the following:
1. Synchronous system
 2. Reliable delivery
 3. No message forgery
 4. Known sender identity

+ 0 pts Incorrect / Unattempted

4.2 (II)

2 / 2 pts

✓ **+ 1 pt** $n \geq 3m + 1$

✓ **+ 1 pt** $n = 7$

+ 0 pts Incorrect / Unattempted

4.3 (III)

0 / 4 pts

- + 1 pt** Recursive elimination of traitors
- + 1 pt** One layer per faulty process
- + 1 pt** Base case OM(0)
- + 1 pt** Majority consistency argument

✓ **+ 0 pts** Incorrect / Unattempted

4.4 (IV)

2 / 3 pts

✓ **+ 1 pt** Recursive expansion

✓ **+ 1 pt** Messages grow combinatorially

+ 1 pt $O(n^{m+1})$

✓ **+ 0 pts** Incorrect / Unattempted

Question 5

Problem 5

0 / 12 pts

5.1 (I)

0 / 6 pts

+ 1 pt $n = 2f + 1$

+ 1 pt Each quorum $\geq f + 1$

+ 1 pt Disjoint $\geq 2f + 2$

+ 1 pt Showed above contradiction

+ 1 pt Showed non-empty intersection

+ 1 pt At least one correct replica ($\leq f$ faulty)

✓ + 0 pts Incorrect / Unattempted

5.2 (II)

0 / 6 pts

+ 6 pts Any 3 of the following:

1. Crash faulty model only
2. No equivocation handling
3. Byzantine can send conflicting messages
4. Quorum intersection insufficient
5. No cryptographic authentication
6. Safety can break

+ 4 pts Any 2 of the following:

1. Crash faulty model only
2. No equivocation handling
3. Byzantine can send conflicting messages
4. Quorum intersection insufficient
5. No cryptographic authentication
6. Safety can break

+ 2 pts Any 1 of the following:

1. Crash faulty model only
2. No equivocation handling
3. Byzantine can send conflicting messages
4. Quorum intersection insufficient
5. No cryptographic authentication
6. Safety can break

✓ + 0 pts Incorrect / Unattempted

6.1 (I)

3 / 3 pts

✓ + 3 pts Correct probability reasoning + correct final answer

Answer: (0.64)

Any 1 of the following:

1. $P(\text{atleastone}) = 1 - P(\text{none})$
2. $P(\text{atleastone}) = P(\text{round1}) + P(\text{round2}) - P(\text{both})$
3. $P(\text{atleastone}) = P(\text{Awinsround1only}) + P(\text{Awinsround2only}) + P(\text{Awinsboth})$

+ 2.5 pts Correct setup, but calculation mistake

Answer: (0.64)

Any 1 of the following:

1. $P(\text{atleastone}) = 1 - P(\text{none})$
2. $P(\text{atleastone}) = P(\text{round1}) + P(\text{round2}) - P(\text{both})$
3. $P(\text{atleastone}) = P(\text{Awinsround1only}) + P(\text{Awinsround2only}) + P(\text{Awinsboth})$

+ 1 pt Correct single-round probability but incorrect combination logic

Answer: (0.64)

Any 1 of the following:

1. $P(\text{atleastone}) = 1 - P(\text{none})$
2. $P(\text{atleastone}) = P(\text{round1}) + P(\text{round2}) - P(\text{both})$
3. $P(\text{atleastone}) = P(\text{Awinsround1only}) + P(\text{Awinsround2only}) + P(\text{Awinsboth})$

+ 0 pts Incorrect / Unattempted

6.2 (II)

0 / 5 pts

+ 1 pt Honest Utility

$$U_H = R$$

+ 2 pts Dishonest Expected Utility

$$U_D = (1 - p)G - p(\alpha S)$$

G is extra gain here. Assuming $G = R$ is also acceptable.

+ 1 pt Rationality Condition

$$U_H \geq U_D$$

+ 1 pt Final Condition

$$p \geq (G - R)/(G + \alpha S)$$

or

$$p(\alpha S + R) \geq 0$$

✓ + 0 pts Incorrect / Unattempted

Part (i)

✓ + 1 pt Determine total required amount
 $TotalRequired = 3 + 0.1 = 3.1BTC$

✓ + 1 pt Select Valid Inputs
 $\sum Inputs \geq 3.1BTC$
Accept any valid combination

Part (ii)

✓ + 1 pt Compute Correct Change
 $Change = Inputs - 3.1$

✓ + 1 pt Clearly State Outputs
 $Recipient, Change\ address$

+ 0 pts Incorrect / Unattempted

7.1 (I)

0 / 10 pts

Setup

One party creates a secret and shares its hash with the other party so it can be used in both contracts

+ 2.5 pts Secret creation and sharing of hash clearly stated

+ 1.5 pts Secret/hash idea mentioned but unclear

Lock

Both parties lock their assets in HTLC contracts using the same hash. The BTC is locked first, then the ETH, with a shorter timeout.

+ 2.5 pts Both locks and timeout relationship correct

+ 1.5 pts Only one lock or missing timeout logic

Claim

The BTC holder claims the ETH by revealing the secret. The other party then uses the revealed secret to claim the BTC.

+ 2.5 pts Both claims described in correct order

+ 1.5 pts Only one claim step described

Refund

If the swap does not complete, each party can recover their locked funds after the respective timeouts.

+ 2.5 pts Refund logic clearly stated

+ 1.5 pts Refund mentioned but incomplete

✓ + 0 pts Incorrect / Unattempted

Same Hash Condition

Both HTLCs use the same hash condition, so the secret needed to unlock one contract is the same for the other.

- + 1.5 pts Clearly explains that both contracts depend on the same hash/secret
 - + 0.5 pts Mentions shared condition but unclear
-

Secret Reveal Links the Claims

When one party claims funds, the secret is revealed on-chain, allowing the other party to claim their funds.

- + 1.5 pts Correct reasoning that the reveal enables the second claim
 - + 0.5 pts Partial explanation
-

Timelock Ordering

The second contract has a shorter timeout, ensuring the first party has time to claim after the secret is revealed.

✓ + 1.5 pts Correct explanation of timeout ordering

+ 0.5 pts Mentions timeouts but reasoning unclear

Refund Safety

If the swap does not proceed, each party can refund their funds after timeout, preventing loss.

+ 1.5 pts Refund logic clearly explained

✓ + 0.5 pts Partial explanation

+ 0 pts Incorrect / Unattempted

Name: Chirag KethpalaEntryNo 2025MCS2038

Total:90

Note Write your answers neatly and precisely. You won't get a second chance to explain what you have written.

Problem 1: 12 Marks

(I) Select all that applies. Nakamoto consensus differs from classical Byzantine Fault Tolerance because (2 Marks)

- ☒ (a) Voting power is proportional to computational work.
- ☐ (b) Node identities must be authenticated
- ☐ (c) Consensus is probabilistic.
- ☒ (d) Participation is permissionless.
- ☐ (e) It guarantees instant finality ~~X~~

(II) Select all that apply. For any options not selected, provide a brief justification for their exclusion. In Bitcoin's transaction (txn) structure: (6 Marks)

- ☒ (a) Txns can have multiple input and output
- ☒ (b) Outputs can be partially spent
- ☒ (c) Each input must reference a previous unspent output
- ☒ (d) Coins exist as independent state objects

(d) ~~Coins~~ Coins depend on prev unspent output so they are not independent

(III) Select all that apply. For any options not selected, provide a brief justification for their exclusion. An attacker attempting to reverse a confirmed transaction must: (4 Marks)

- ☒ (a) Outpace honest miners in cumulative PoW
- ☒ (b) Control more than 50% of network nodes
- ☒ (c) Build an alternative chain from the attack point
- ☐ (d) Obtain private keys of honest users

(d) he doesn't need it, as in he can just go to that point, start new chain and get it to be more than current running chain

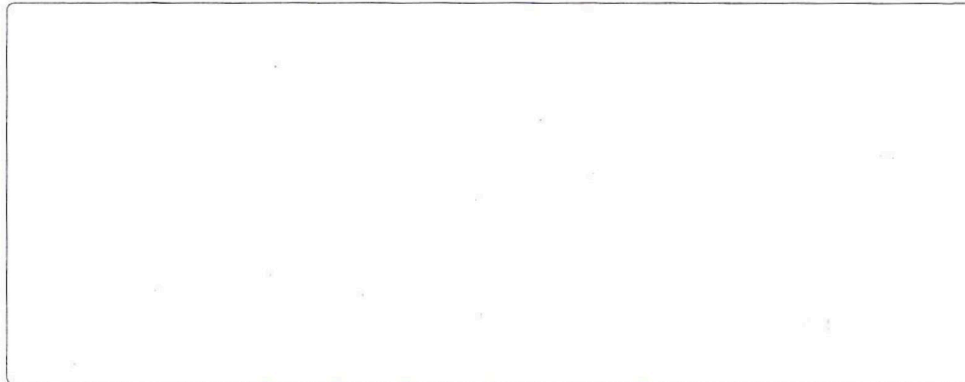
Problem 2: 16 Marks

- (I) Consider the method of combining the signature and encryption (Sign-and-Encrypt): $C = E_{PK_B}(SSK_A(m))$. The symbols here are self-explanatory. This scheme has an attack that allows a legitimate recipient, say B , to convince a third party, say C , that A sent a message to C even though A only intended it for B . Explain the attack in steps. (4 Marks)

Attacker can actually decrypt the message using his (B) private key which leads to [message + sign(A)] to which he can encrypt with public key of C and forward to C , thus when C try to unlock it with its private key it would look like it was sent by A as it has signature of A which A but in when sending to B .

- (II) Consider an alternate method (Encrypt-and-Sign): $C = (E_{PK_B}(m), SSK_A(E_{PK_B}(m)))$. This resolves the forwarding attack described in the previous question. Explain how. (4 Marks)

So this Signature is Done using Private key of A that is only known to A and to actually do the attack the Receiver must first use Public key of A to figure out it was sent by A and then on the encrypted message use its own private key (of B) to read it, since B can not recreate private key of A it will not be able to send it to C as if it sends what it received C won't be able to read message as its encrypted by public key of B and can only be decrypted by private key of B , Also, since C receive it from B it would think its encrypted by B and try its supposed to try public key of B to open it thus never receiving message.



(III) A scheme (E, D) is IND-CCA (indistinguishable under chosen-ciphertext attack) secure if no PPT adversary $\mathcal{A}$ can win the following game with probability non-negligibly greater than $\frac{1}{2}$:

- $\mathcal{A}$ is given the public key and access to a decryption oracle $\mathcal{O}_D$
- $\mathcal{A}$ submits two messages m_0, m_1 of equal length
- Challenger encrypts $C^* = E(m_b)$ for a random bit b
- $\mathcal{A}$ may query $\mathcal{O}_D$ on any ciphertext except C^* itself
- $\mathcal{A}$ outputs b' ; wins if $b' = b$

Show *Sign-and-Encrypt* protocol does not achieve IND-CCA. You can use textbook RSA for (E, D) scheme. That is, encryption is $C = m^e \bmod N$ and signature is $m^d \bmod N$, where RSA public key is (N, e) and d is the private key. (8 Marks)



Problem 3: 10 Marks

- (I) Explain three differences between Multi-signature schemes and Threshold signature schemes. (3 Marks)

<u>Multi-signature</u>	<u>Threshold signature scheme</u>
① Gas cost = $O(n)$	① Gas cost = $O(1)$
② Final encryption is encryption of one on another for all member parties	② Final encryption is concatenation of All member parties
③ Require Signature of All n	③ can work on <u>Signature</u> of less k outta n

- (II) In Shamir's secret sharing scheme with $k = 3$, what is the degree of the polynomial used? Why $k - 1$ participants learn nothing? (3 Marks)

It requires key from atleast k people to ~~decrypt~~ get the message, at less than k the message never ~~dec~~ decrypts
 degree of polynomial used = ~~$O(n)$~~ $O(n^k)$

- (III) Explain each step involved in obtaining a proof-of-non-membership from a Merkle tree. (4 Marks)

All the nodes hash are stored as leafs in sorted order, to prove a ~~transaction~~ ^{transaction} is not in the leaf we can use binary search and search for its hash

Problem 4: 12 Marks

- (I) In the class we discussed Lamport Shostak Pease "Oral Messages" protocol $OM(m)$ for Byzantine agreement. List assumptions on the fault model, network model, and network links for the protocol to work correctly. (3 Marks)

① Network model is peer to peer
 ② fault model is Byzantine nodes (Remain Silent & modify)
 ③ network links are one to one
 what happens is at each iteration a ~~total~~ node becomes leader and sends to all nodes what it heard later, when all are done then we ~~go~~ make decision based on inputs received, N/w may get stuck if a node doesn't respond

- (II) You want to tolerate $m = 2$ faults using $OM(2)$. What is the minimum n ? Explain your answer. (2 Marks)

Min size of $n = 3m + 1 \Rightarrow 3 \times 2 + 1 = 7$
 To make a decision it needs failure nodes to be at least less than $n/3$ $[f < n/3]$

- (III) Explain in bulleted points, why $m+1$ rounds are necessary in the $OM(m)$ protocol. (4 Marks)

- Total Rounds is $2m+1$
- if $m+1$ nodes fail consecutively we are sure that we can not make a decision
- fault model used in O.M. require consensus of Atleast ~~m~~ $m+1$ or rather $>33.33\%$ of N/w

- (IV) Derive the message complexity of the $OM(m)$ protocol on n nodes. Explain each step. (3 Marks)

- Gossip phase →
- ① one response is given by commander to All
 - ② each Lieutenant takes its turn act as commander and broadcast its message (what it heard)
 - ③ Evaluation of each Lt throughout the Gossip phase
- T.C = $O(n^m)$ each Lt sending broadcast to All takes $O(n)$ and this is done recursively atleast m times

Problem 5: 12 Marks

- (I) In view stamped replication (VSR) protocol, each quorum (a quorum is the minimum number of participating replicas required to perform an operation successfully and ensure the system remains consistent) must consist of at least $f + 1$ replicas. Prove that two VSR quorums must intersect in at least one correct replica. (6 Marks)

- (II) Why can VSR not tolerate Byzantine faults even if $n = 3f + 1$? (6 Marks)

VSR works on state machines ~~this~~ thus it can take ~~max~~ $f < n$ faults, as long as atleast 1 works it works

Problem 6: 12 Marks

- (I) In a PoS blockchain, validator selection probability is proportional to stake. Assume A has 40% of the stake of the total stake. What is the probability that A validates at least one block in two consecutive rounds. (3 Marks)

$$\begin{aligned}
 &\text{Prob that } A \text{ gets at least 1 block} \Rightarrow \\
 &\Rightarrow \text{Prob of getting in 1st} + \text{Prob of getting in 2nd} \\
 &\quad \quad \quad \text{fail in 1st, pass in 2nd} \\
 &\Rightarrow 0.40 + 0.60 \times 0.40 \\
 &\Rightarrow 0.40 + 0.24 \Rightarrow \underline{\underline{0.64}}
 \end{aligned}$$

- (II) Suppose a validator with stake S earns reward R per honest block but lose fraction αS if caught signing conflicting blocks. Assume probability of getting caught is p . Derive condition (as an under which rational validator behaves honestly. (5 Marks)

So if he ^{gets caught} with prob p thus if there are n rounds np times he gets lost and $(1-p)n$ times he get reward R . To ensure he doesn't cheat he should lose more than he ~~earn~~ earns.

So in np times he should lose more than $(1-p)nR$

but he lose his ~~stake~~ stake not in R

in 1 time if he lose αS in np time he lose $np\alpha S$

$$\text{thus } \alpha S \text{ should be } \leq \frac{(1-p)nR}{np}$$

to ensure he doesn't cheat

(III) A user owns the following UTXOs: 2.5 BTC, 1.2 BTC, 0.8 BTC. They must send 3 BTC with fee 0.1 BTC. (i) Select valid inputs (ii) Specify transaction outputs. (4 Marks)

Valid inputs -> 2.5 BTC, 1.2 BTC
→ 2.5 BTC, 0.8 BTC

The transaction would send coins, charge fees and return rest so

In 1st it would send 0.6 BTC back
in 2nd it would send 0.2 BTC back

Problem 7: 16 Marks

Aarti holds 1 BTC and Bharti holds 20 ETH. They want an atomic swap $\text{BTC} \leftrightarrow \text{ETH}$ using HTLCs.

(I) Write the complete protocol steps - setup, lock, claim and refund. (10 Marks)

- (II) Provide reasoning for the swap to never allow "only one side paid" type of terminal state if both follow the protocol. (6 Marks)

So whoever starts the transaction has to set his timeout greater than the other
say A start transaction to B
A's Timeout $>$ B's timeout
thus if A claims B's coin and B doesn't in the specified time they both get returned
 $T_1 > T_2$ — this ensures the one who started can't cheat and get other parties coins

Rough Page